Environmental Monitoring Using Low-Cost Hardware and Infrastructureless Wireless Communication

Lars Baumgärtner*, Alvar Penning*, Patrick Lampe*†, Björn Richerzhagen†, Ralf Steinmetz†, Bernd Freisleben*†,

*Philipps-Universität Marburg, FB 12, D-35032 Marburg, Germany
E-mail: {lbaumgaertner, penning, lampep, freisleb}@informatik.uni-marburg.de
† Technische Universität Darmstadt, FB 18, D-64289 Darmstadt, Germany
E-mail: {bjoern.richerzhagen, ralf.steinmetz}@kom.tu-darmstadt.de

***Abstract*—Infrastructureless wireless sensing is beneficial for environmental monitoring in various application scenarios, such as in a disaster where it is vital for the operational planning of professional responders, or in documenting long-term changes in climate and biodiversity in natural heritage sites. In this paper, we present a low-cost yet flexible and powerful hardware/software platform for sensing, computation, and infrastructureless wireless communication. Since energy consumption is a key issue for autonomous operation, we investigate the power requirements of various computing platforms, sensors and radio link technologies. Furthermore, we experimentally evaluate several license-free radios (WiFi, Bluetooth, LoRa 433 MHz and 868 Mhz) in terms of their real-world communication ranges. To avoid wasting precious communication resources, we apply machine learning and image-based concept detection on-device to remove irrelevant data before sending it. We also evaluate neural compute sticks to improve performance and optimize their energy consumption in our specific scenario. The proposed hardware/software platform consists of small, low-power sensor nodes based on microcontrollers, larger single board computer relay nodes that can also preprocess data, and Bluetooth Low Energy enabled smartphone add-ons to give mobile devices access to long range communication technology. Finally, a Linux distribution tailored to the specific needs in this application area is presented that makes deploying new relay nodes easy even for non-specialists.**

I. Introduction

Environmental monitoring without being able to rely on existing electricity and communication infrastructures is required in several use cases, such as biological and ecological studies (e.g., animal tracking, air quality measurement), delivering Internet of Things (IoT) technology to farms[1] in rural places, and emergency communication in disaster scenarios (e.g., tsunamis, earthquakes, nuclear meltdowns). In these situations, several constraints need to be considered when a technical solution for environmental monitoring is designed: lack of power supply, lack of communication infrastructures, harsh weather, low maintenance possibilities, weight restrictions for animal-attached sensors, availability of scenario-specific sensors, and cost factors.

There are several computing platforms readily available that can be extended to provide stationary services as well as mobile, low power tracking devices. Furthermore, there are many affordable, off-the-shelf sensors that are potentially useful in such scenarios, but proper information on how to integrate them and their specific requirements, such as real world energy consumption, are often not easily accessible. Moreover, integrating the computing power, flexibility and mobility of smartphones and tablets is an interesting option. Since satellite communication is expensive and limited, and cellular services are often not available in remote places or during a disaster, low-cost long-range (up to 16 km) radio technologies, such as LoRa, are quite useful. Although they are mostly associated with IoT applications in an infrastructure-based LoRaWAN mode, they can also be used for device-to-device communication and have the benefit of license-free frequency bands in any country of the world. Due to the low bandwidth of these radio transceivers, only small pieces of data can be transmitted, increasing the need to preprocess data prior to long-range transmissions.

In this paper, we present a flexible and affordable sensor, computation, and communication platform for environmental monitoring that relies on low-cost hardware and infrastructureless communication. It uses delay-/disruption-tolerant networking (DTN) for non-time critical tasks over different wireless links, provides on-device data processing capabilities based on machine learning methods, and integrates mobile sensor nodes, static devices, and smartphones. In particular, we make the following contributions:

- We present a novel sensor, computation, and communication platform for static and mobile environmental monitoring setups, as well as users with mobile devices.
- We present a novel energy-efficient approach for on-device image classification.
- We present a novel approach to provide long range communication capabilities for Bluetooth-enabled devices.
- We present experimental evaluations of (a) commonly available and affordable platforms, (b) the energy consumption of several sensors, and (c) the real world communication ranges and energy demands of various radio link technolgies for environmental monitoring.

[1] https://wazihub.com

The paper is organized as follows. Section II discusses related work. In Section III, we present requirements and designs decisions. Section IV discusses implementation issues. Section V presents experimental results. Section VI concludes the paper and outlines areas of future work.

II. Related Work

Several wireless sensor platforms for environmental monitoring have been presented in the literature, but they are often either based on specific technologies or tailored to dedicated use cases. For example, the OpenSense project aims to bring community-driven environmental monitoring to life with an open platform and accessible results [1]. However, it is specifically designed for air pollution monitoring, relies on existing communication infrastructures and hardware specifically built for this use case. On the other hand, it integrates static sensor nodes as well as mobile nodes, and most recently also wearables [2]. Similarly, Citi-Sense-MOB [3] and AirSenseEUR[2] [4] work for air quality measurements in urban environments, but also rely on technologies such as GRPS for data transmission, constant power supply (e.g., from the bus or car the sensor is mounted on), and custom/closed-source PCBs.

Some custom-built platforms can be adopted to various scenarios. Problems associated with custom-built platforms are their cost and availability; sometimes they are specifically tailored to a particular geographic region [5], [6].

Llamas et al. [7] use Arduinos and Raspberry Pis for building a reusable open sensor platform. Their application is human gait identification. Long range communication or infrastructureless operation are not considered.

The senseBox project[3] provides a sensor platform for citizen science projects [8]. There is a strong focus on education and publicly available sensor data, but senseBox lacks flexibility when used for more than just raw sensor data aggregation and also relies on existing communication infrastructures. Luftdaten[4] [9] also uses cheap MCUs configured for air quality measurement in a citizen science project, but also relies on existing communication infrastructures.

The EU project WAZIUP tries to bring IoT technologies to developing countries [10]. WAZIUP provides a low-cost communication infrastructure (LoRaWAN), while we try to work without any infrastructure on a purely peer-to-peer basis. We incorporate DTN technologies and feature heterogeneous radio-link setups. Furthermore, conserving energy plays a more vital role in our use cases, and we focus on providing a complete system for remote sensing and communication.

Some proposals use mobile devices or wearables as sensors, and some approaches provide low-cost, long-range radios usable from smartphones, but they are often very impractical (a direct USB connection to a smartphone is required) or require operating system modifications [11].

[2]https://airsenseur.org/website/

[3]https://www.sensebox.de

[4]https://www.luftdaten.info

III. Sensor Platforms for Environmental Monitoring

Static Sensor Platforms (SSPs) are the backbone of our environmental monitoring platform. They have less restrictions on size or weight than Mobile Sensor Platforms (MSPs), and energy problems can be handled easier by adding solar panels, wind turbines, or larger batteries. SSPs can be used for relaying packets, collecting sensor data on roof-tops, on trees (e.g., wildlife cameras), or on smart street lamps. MSPs, on the other hand, are used to tag animals and, therefore, must be kept as light and small as possible. Energy is a more valuable resource. Since humans with smartphones and tablets can also act as sensors, these must be integrated as well. Adding long-range infrastructureless communication to mobile devices also has the benefit that it can be used in case of an emergency for sending messages to other participants. Finally, it is necessary to preprocess the gathered sensor data to reduce the needed bandwidth for transmission. This is challenging when using power-efficient devices as a basis for SSPs. The different sensor platforms are discussed in more detail below.

A. Static Sensor Platforms

SSPs can easily utilize different energy sources, such as solar panels, car batteries, or electrical wall outlets. Therefore, they can be built using regular Single-Board-Computers (SBCs), such as Raspberry Pi 3 or Zero, since these platforms offer a compromise between computational power and low energy consumption. To also function as a network hub for mobile users and MSPs, different radio link technologies can be incorporated - Bluetooth, WiFi mesh, and for longer range, license-free, low-bandwidth communication via LoRa. Since energy consumption of SSPs is not as critical as in MSPs, SSPs can constantly listen on the different wireless interfaces and act as hubs or relays for smaller mobile nodes passing by. For data dissemination, DTN technologies such as Serval[5] can be used, since they perform well in infrastructureless environments [12]. A variety of sensors can be added, e.g.:

- camera, night-vision camera, thermal camera
- microphone
- GPS/GLONASS
- temperature, humidity, barometric pressure
- air quality
- rain- and soil-moisture sensors

Usually, these are connected directly through GPIO pins or various bus systems (e.g., serial, SPI, i2c). Most SBCs directly provide these interfaces, only Analog-Digital pins are often lacking, but can easily be added externally.

B. Mobile Sensor Platforms

MSPs have more constraints than SSPs, since they must be as light and small as possible to be attached to animals or additionally mounted on an Unmanned Ground or Aerial Vehicles (UGV/UAV) [13]. Apart from the weight of the total system, power consumption is the main bottleneck. Due to these limitations, the choice of radio link technologies and

[5]https://github.com/servalproject/serval-dna

the types of sensors are quite restricted. MSPs are based on small microcontroller units (MCUs), since they require much less power and often provide deep sleep capabilities as well as various I/O pins for digital and analog sensor inputs.

C. Long-range Radio Links for Mobile Devices

In remote areas, cellular coverage or WiFi access points for communication using standard smartphones are often not available. Building custom radio links into phones is quite expensive and economically not interesting for large carriers. Therefore, we need a different approach with the following requirements:

1) it should work with any operating system,
2) it should be compatible with any mobile phone,
3) it should be license-free "worldwide",
4) it should provide infrastructureless long-range (>1 km) communication,
5) it should be energy-efficient,
6) it should be affordable.

The best solution for items 1 and 2 is to rely on platform-agnostic standards such as Bluetooth Low Energy (BLE) that can be used from Apple iOS and Google Android, or almost any other operating system. To cover items 3 and 4, we use the LoRa standard that in theory provides up to 16 km of range and is available in different frequency bands (433/868/915 MHz) for worldwide usage. LoRa also offers the LoRaWAN standard as a higher layer with built-in encryption, but it also requires some infrastructure, making it unsuitable for our task. Energy efficiency (item 5) is partly achieved by using BLE, but also by using small MCUs to handle communication. By finding a suitable MCU with built-in Bluetooth and LoRa chipsets, the price for such a smartphone add-on is also kept low. Alternatively, an SBCs equipped with a LoRa modem can be paired using Bluetooth with a smartphone. Even though energy consumption will be higher, this system has the benefit that software such as Serval can run directly on the SBC and expose only UI relevant functions via BLE, preserving smartphone energy due to lesser notifications and wake-ups.

D. On-Device Data Processing

Since long-range links over LoRa only provide a few kilobits per second of bandwidth, transmitting large amounts of data in its raw form is not feasible. Therefore, processing the sensor data at least partially on the SSP to filter out unwanted content or already produce analysis results is favorable. In our use cases, we heavily rely on visual object/concept detection in images to filter out relevant information. Our previous work has shown how this approach can significantly reduce the amount of data necessary to be transmitted [14]. To cope with the limited CPU power of most SBCs, we integrate external hardware to accelerate the visual classification process. The challenge is to limit the power consumed by the accelerator device when it is currently not in use.

IV. Implementation

We now discuss implementation details of our platforms.

Fig. 1. Base station for monitoring, relaying and processing.

A. Static Sensor Platform

The SSP can act as a relay for MSPs and mobile devices carried by users. Furthermore, it can be used without any sensors as a base station and uplink to the Internet (see Fig. 1). To provide these features, we focused on ARM-based SBCs, more specifically the Raspberry Pi family. These devices are readily available worldwide, are proven technology made specifically for tinkerers and well documented. There are several cameras that can be directly attached, plus many (p)HATs with additional hardware and many GPIO pins for direct sensor attachment. Depending on particular requirements (size, energy etc.), there are different models available, such as the Pi Zero W and the Pi 3. Many wildlife cameras and weather stations have already been built on this proven hardware platform, and Raspbian provides a solid and familiar Linux foundation. We modified a Raspbian OS Image to provide features specific to our SSP. It can easily setup a mesh network, provide Bluetooth debugging capabilities, open an access point, and start services, such as GPS logging and serval-dna. Also, the energy consumption can be optimized by deactivating unused features, such as the HDMI port of the Raspberry. All this can be configured through a simple text file on the small FAT32 partition on the SD card[6]. This has the benefit that the default image can be written to a new SD card and then can be configured from any computer with a simple text editor prior to actually booting the system. There is no need for Linux knowledge or extra drivers to read the filesystem, all is kept in one file, in one place for ease of use. The single biggest feature missing in the Raspberry Pi family is proper power management and deep sleep, but there are several after-market solutions[7][8] offering this functionality. Another benefit of the newer Pi's (Zero W(H) and 3) is that Wi-Fi and Bluetooth are already present. The only major interface left to be added for our system is a LoRa transceiver for long-range communication.

[6]https://github.com/buschfunkproject/heckenschere
[7]https://spellfoundry.com/product/sleepy-pi-2/
[8]http://www.uugear.com/witty-pi-realtime-clock-power-management-for-raspberry-pi/

LoRa Radio Modem: There are several MCUs on the market using Cortex M0, ATmega32u4, or ESP32 chips with onboard rfm95 transceiver modules. Using the RadioHead library[9], these can easily be used in the Arduino programming environment to send packet data from device to device. We developed a firmware to expose this functionality via an *AT* command set over the built-in USB-Serial, similar to the one found in classic dial-up modems. Therefore, any device with an USB port can use such a modem, and no special drivers are needed. The source of the modem firmware can be found on github[10]. Furthermore, Serval, our DTN middleware, was made aware of this communication channel by changing LBARD and writing a driver for our firmware[11].

B. Mobile Sensor Platform

Sine the MSP should be as light and small as possible, we focused on MCUs with integrated Wi-Fi and/or LoRa transceivers. For convenience, performance and portability, all of our programming was done using the Arduino programming environment in contrast to ESP's IDF or MicroPython. Hence, only minimal changes were necessary to switch from one MCU to another. To preserve battery power, the mobile sensors are mostly kept in deep sleep until a timeout or external trigger wakes them up. Also, they are programmed as send-only devices, since relaying data would require constant listening and would prevent deep sleep, thus draining the battery faster.

C. Long-range Radio Links for Mobile Devices

Since newer SBCs, such as the Raspberry Pi 3 and Pi Zero W, provide Bluetooth capabilities and can control devices with our radio modem firmware, such a setup can easily be used as a bridge for smartphones. For a simple prototype, we used the bleno framework[12] to expose system functionality via BLE. This is also useful for debugging head-less systems. Therefore, we provide a simple echo service to broadcast any information via BLE[13] from our customized Raspbian distribution. Despite the flexibility this setup provides, it consumes quite a lot of energy and is rather large in size (Pi Zero + LoRa modem + battery pack + antenna).

For a more lightweight solution, we enhanced the radio modem firmware to directly use BLE as a serial UART replacement. The packet size restrictions of LoRa and the BLE implementation in most RF95-based chipsets are quite similar. There are ESP32-based MCUs, such as the ones from Heltec and TTGO, that provide WiFi, Bluetooth and LoRa on a single board. The resulting system can be paired with any BLE capable (mobile) device, requires much less power, and for the 868/915 MHz bands, a rather short antenna. A modified version of the firmware can also be found online[14]. This system is rather small and comparable to commercial products, such as the GoTenna[15], but more open and flexible. The BLE LoRa modem based on a TTGO ESP32 chip, including a small 750 mAh LiPo, a custom 3D printed case, and a 868 MHz antenna is shown in Fig. 2 right next to a Raspberry Pi 3 for a size reference.

Fig. 2. BLE LoRa modem with plain Raspberry Pi 3 for size comparison.

D. On-Device Data Processing

For data processing on SBCs, we implemented a solution with two execution options. The first one is using the device CPU itself, and the second one utilizes a Intel® Movidius™ Neural Compute Stick (NCS). In case we use the compute stick, the CPU of the SBC can go to deep sleep and save energy. Visual object/concept detection is used for image processing, and for this purpose we use two neural network models. The first one is InceptionNet v3 [15] with 1001 different detectable classes, and the other one is also an InceptionNet v3 model trained on the Open Images Dataset[16] This neural network can detect 6012 different classes and is suitable for a more detailed analysis of the given images.

These neural network models are executed either with Tensorflow 1.1.0 built for ARM CPUs or on the Movidius compute stick. Additionally, the neural networks have to be converted with the Movidius compiler *mvNCCompile* to run the pretrained versions on the compute stick.

During our evaluation we found that the Intel Movidius NCS uses an average of 392 mA when it is idle. On a Raspberry Pi, we can disable the entire USB subsystem, but in this case the Ethernet and all other USB ports are also disabled. Since various sensors and potential extra radio link interfaces might be connected over USB, this behavior is not desirable. Disabling only the specific port of the NCS to save energy when no processing has to be done is more favorable. To achieve this, hub-ctrl[17] is used to turn off the power of the compute stick in case it is currently not needed. This preserves a lot of energy, since the compute stick is expected to have more idle times than compute times. The possibility to have the compute stick as a backup without an energy penalty and only activating it when really needed makes it even more useful.

[9] http://www.airspayce.com/mikem/arduino/RadioHead/
[10] https://github.com/gh0st42/rf95modem
[11] https://github.com/gh0st42/lbard-ng
[12] https://github.com/noble/bleno
[13] https://github.com/buschfunkproject/bledebug
[14] Code will be released with the camera-ready version of this paper.
[15] https://www.gotenna.com
[16] https://storage.googleapis.com/openimages/web/index.html
[17] https://github.com/codazoda/hub-ctrl.c

TABLE I
OVERVIEW OF SBC AND MCU PLATFORMS

	Processor	**RAM**	**Network**	**Idle** - Power in A		**Load** - Power in A	
				Mean	Max	Mean	Max
Pi 1 Model B+	1x 700 MHz	512 MB	100 Mbit Eth.	0.187	0.261	0.210	0.290
Pi 3 Model B	4x 1200 MHz	1024 MB	100 Mbit Eth., WiFi, 4.1	0.224	0.383	0.474	0.734
Pi Zero W	1x 1000 MHz	512 MB	WiFi, 4.1	0.082	0.139	0.131	0.213
ESP32, TTGO	1x 240 MHz	520 KB	WiFi, 4.2	0.072	0.084		
ESP8266, Feather Huzzah	1x 80 MHz	80 KB	WiFi	0.084	0.261		
Feather M0	1x 48 MHz	32 KB	Optionally WiFi, LoRa	0.005	0.006		
Feather 32u4	1x 8 MHz	2 KB	Optionally LoRa	0.005	0.005		
Waspmote	1x 14.74 MHz	8 KB		0.029	0.030		

The visual object/concept detection software itself was written in Python 2.7 and outputs the five main visual concepts for each given image for further processing or discarding of irrelevant images. Thus, the energy for transmitting irrelevant images can be saved. Especially in a multi-hop scenario, a large amount of energy can be saved by applying preprocessing functions to recorded image data [16].

V. EXPERIMENTAL EVALUATION

We performed several experimental evaluations regarding power consumption, transmission ranges, and execution times. We used the Monsoon Power Monitor for power measurements. All energy tests were repeated 2-3 times and ran for about one minute each, resulting in plenty of time to get a realistic energy reading. Only the stress and compute stick evaluations were not terminated by time but by the given task.

A. Platform Comparisons

We evaluated some Raspberry Pi SBCs as well as MCUs from different vendors. They were tested for their idle power consumption and under full load. Other chipset-specific features such as RAM, onboard networking capabilities etc. are also listed in Table I. When looking at the different Raspberry Pi models, it is evident that when idling, the Raspberry Pi Zero W (82 mA) only consumes half of what a Pi 1 (187 mA) needs and just about a third of what the base Pi 3 Model B (224 mA) needs. The newest Pi 3 Model B+ was not tested, but due to its increased CPU power it is expected to consume even more power. Both Pi 1 and Pi 3 offer more USB ports onboard and Ethernet. The Pi 3 offers more processing power and cores than the relatively power-hungry Pi 1 that provides even less processing power than the Pi Zero W. Under load, the Pi Zero W still consumes less power than a Pi 1 idle. Due to its 4 cores and high clock speed, the Pi 3 uses almost 500 mA under load and peaks to over 700 mA at certain times. Thus, if saving power is the priority, then the Pi Zero W is the best choice. If multiple CPU cores, onboard Ethernet, and more USB ports are essential, then the Pi 3 Model B is the best candidate.

For our MSP we evaluated several MCU candidates, as shown in the lower part of the table. Some of them like the ESP32 board from TTGO are much more powerful with 240 MHz and 520 KB RAM compared to systems like the Waspmote with 14.74 MHz and 8 KB RAM. Not all systems provide deep sleep functionality by default. Even though the Waspmote or Feather M0 system consume less power than the ESP32, they also lack the radio link interfaces. Due to the flexibility through the included radio interfaces, cost of the board and provided CPU/RAM as well as deep sleep capabilities and I/O pins, we mainly built our MSPs on ESP32 boards. Should RAM not be an issue and a single radio-link interface be sufficient, the Feather M0 or even the low-power 32u4 are good alternatives with minimal power requirements.

B. Evaluation of Sensor Requirements

We measured the typical power requirements of different sensor types, as shown in Table II. The sensors range from simple temperature and particle sensors over GPS receivers to complex optical systems such as (night vision) cameras.

TABLE II
OVERVIEW OF SBC AND MCU PLATFORMS

	Function	**Power** in A	
		Mean	Max
MAX30105	Particle	0.004	0.029
CCS811	VOC, TVOC, eCO2	0.010	0.037
PIR	PIR	0.010	0.011
SIM28 GPS	GPS	0.049	0.060
BME280	Temp., Baro., Humidity	0.011	0.013
MICS-2714	NO2	0.008	0.009
MICS-5524	CO	0.008	0.012
AMG8833	IR thermal	0.038	0.511
Envirophat	Temp., Baro., Color, Accel., Magnetometer	0.038	0.430
Pi Camera NV	Night Vision Camera	0.270	0.511
Pi Camera v2.1	Camera	0.086	0.511

Most of these sensors consume an average of about 10 mA or less. A notable exception is GPS with about 49 mA and the cameras with 86 mA / 270 mA. It is also noteworthy that the night vision camera draws 3 times as much power as the regular one. A simple 8x8 thermal imaging sensor, such as the AMG8833, on the other hand, only draws an average of 38 mA. When adding these sensors to an MSP or SSP, we must also consider the maximum peaks. These can go up to 511 mA for the optical sensors.

C. Radio Link Comparisons

We evaluated the power consumption of different MCUs with onboard radio transceivers, ranging from Wi-Fi to LoRa,

TABLE III
POWER CONSUMPTION OF DIFFERENT MCUS INCLUDING RADIO TRANSCEIVERS

	Power in A	
	Mean	Max
ESP32, TTGO, WiFi	0.117	0.272
ESP8266, Feather Huzzah, WiFi	0.080	0.424
Feather M0, WiFi	0.094	0.313
ESP32, TTGO, LoRa	0.046	0.163
Feather M0, LoRa	0.009	0.104
Feather 32u4, LoRa	0.009	0.112

Fig. 3. Waterproof static sensor box deployed.

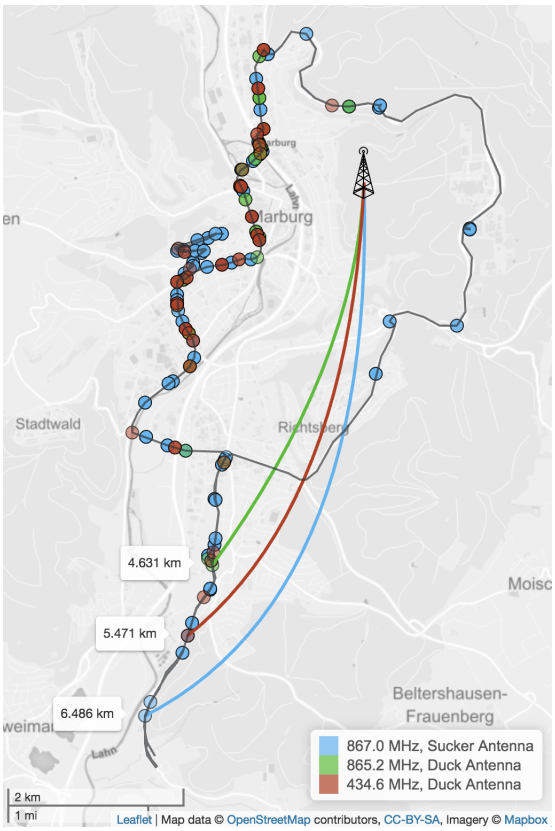

Fig. 4. Communication range of different LoRa setups.

as shown in Table III. This table shows the current draw from the MCUs with included radio transceivers. It is obvious that Wi-Fi communication is at least twice as expensive as LoRa (TTGO ESP32), and in case of the Feather M0, 10 times as expensive. There is also a bigger difference regarding LoRa power consumption when comparing both Feather devices (9 mA) to the TTGO ESP32 (46 mA). Furthermore, sending via LoRa is much more expensive at roughly 100 to 160 mA. If more RAM and CPU power is needed or if more than one radio transceiver with small physical dimensions is required, than the TTGO is the best choice. The Feather sticks are available only either with Wi-Fi or LoRa onboard.

To evaluate the LoRa transceiver modules' communication range, we deployed our sensor box (Fig. 3) and measured the transmission range using one of our GPS modules. We used ESP32 TTGO modules in various configurations. One chip was tuned to the 433 MHz band and equipped with a small wire antenna from the supplier. The next one was set to work in the 868 MHz band, also with the short vendor-supplied antenna. Finally, we set up another TTGO in the 868 MHz band but on a different channel with larger 3.5 dBi magnetic sucker antennas on both ends. The sending interval chosen for the fixed station was set to 7 seconds for each device.

The city of Marburg is surrounded by many hills and forests, therefore, the sending station was deployed on a viewpoint near the university with line-of-sight to most parts of the city. The receiving nodes were mounted on the dashboard of a car, and we drove around the campus, through the city, up to the castle on the opposite side of the city, and then along the valley, until we lost the signal. The route we took, including received beacons, is shown in Fig. 4.

The longest distance covered was almost 6.5 km with the 3.5 dBi sucker antenna. The 433 MHz setup and the other 868 MHz node reached 1-2 km less, but with a much higher packet loss. Also, there was no more line-of-sight from either maximum distance points, it was blocked by hills, houses and trees. The 1.5-2 km distance to the downtown area was covered by all three devices despite trees and houses in the way.

The RSSI over distance is shown in Figure 5, where up to 2-3 km all devices still received signals quite often. As expected, the 3.5 dBi antenna has the best reception throughout the test. This is especially clear for distances over 3 km where the duck antennas both fall short. The total number of received packets in percent can be seen in Fig. 6. The clear winner here, again, is the 3.5 dBi sucker antenna. Both stock antennas from TTGO perform very similar with receive rates between 10% and 15% during our test. Due to our constant movement and the diverse topographic areas, these numbers should be seen in relation to the sucker antenna, not as absolute numbers. Traffic might be responsible for remaining longer in areas where no reception was possible, resulting in a higher overall packet loss rate.

D. On-Device Data Processing

For on-device data processing, we used 1481 images for CPU and neural compute stick execution. Each of these images is 3000 x 2000 pixels in size, since 6 mega-pixels are a good compromise between storage space and visual details. This

TABLE IV
COMPARISON OF CONCEPT DETECTION ON PI VS. NEURAL COMPUTE STICK

	Idle - Power in A			**Starting** - Power in A			**Analysis** - Power in A		
	Mean	Max	Time	Mean	Max	Time	Mean	Max	Time per Image
Pi 3b, 1K, CPU	0.224	0.383	59.29 sec.	0.526	0.579	19.66 sec.	0.546	0.709	13.27 sec.
Pi 3b, 1K, Movidius	0.392	0.420	59.76 sec.	0.475	0.578	3.92 sec.	0.553	0.679	0.61 sec.
Pi 3b, 6K, CPU	0.224	0.383	59.29 sec.	0.533	0.642	75.03 sec.	0.575	0.709	5.46 sec.
Pi 3b, 6K, Movidius	0.392	0.420	59.76 sec.	0.503	0.577	12.64 sec.	0.664	0.735	0.82 sec.

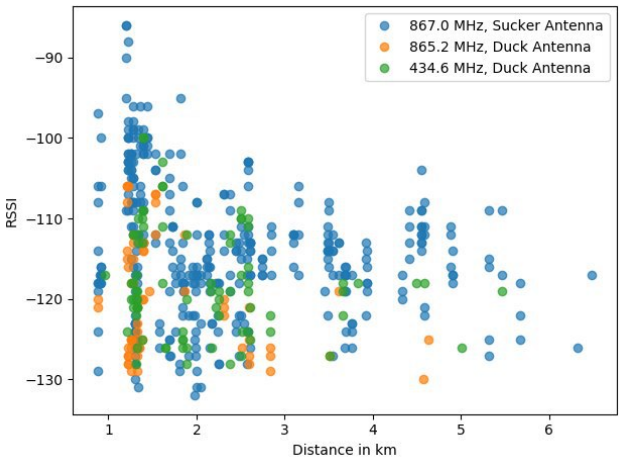

Fig. 5. RSSI vs. Distance

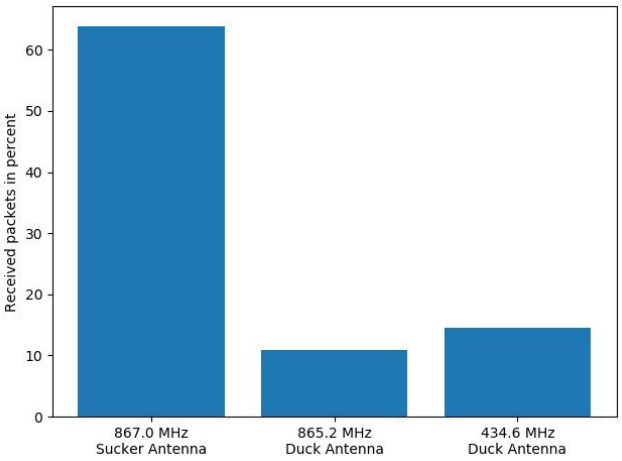

Fig. 6. Number of received packets in relation to packets sent.

dataset is described in our previous work [14]. For evaluation purposes, we built a Python tool to batch process these images. First, the program loads the necessary libraries and the pretrained neural network, then the images are processed. The consumed power and time for the tasks is shown in Table IV.

First, we compared the idle power consumption of the Pi 3 to one with an attached neural compute stick. While the max peak is almost identical, the average current draw is nearly double when a NCS is added. In the starting phase, the neural network is loaded either into RAM for the plain Pi 3 or onto the NCS. Both setups consume around 500 mA with a slight edge for the NCS setup. Taking runtime into account, the compute stick is clearly more efficient by loading the trained model 5-6 times faster. When performing the actual visual concept detection, the mean current is between 546 mA and 664 mA and peaks at 735 mA, with a slightly lower power demand by the CPU setups. Given the much longer runtimes of the CPU version, 6x and 21x slower per image, the NCS is still much more efficient, in terms of speed and energy. The main drawback of the tested compute stick is its high current draw when idle. This problem was solved by deactivating the specific USB port when the stick is currently not needed, reducing the power consumption overhead to zero.

E. Setup Cost

Since we want to provide affordable solutions that can easily be customized to specific needs, we also provide a short overview of the cost of our systems. All prices are in (rounded) Euro, taken in June 2018 from Amazon, Aliexpress etc. All our cases are custom printed on a 3D printer. We also did successful deployments with cheap waterproof junction boxes from the local hardware store (<10 €).

The cheapest system is our BLE LoRa Modem with a total system cost of about 14 € (Table V). A 750 mAh battery lasts roughly between half a day and a full day. This component can easily be swapped for a bigger battery or powerbank.

TABLE V
COST OF BLE LORA MODEM

Part	**Estimate Price**	**Comment**
TTGO LoRa SX1276 ESP32	9 €	incl. antenna
LiPo Battery 750mAh	3 €	
3D printed case	2 €	
Total Price	14 €	

TABLE VI
EXAMPLE CONFIGURATION FOR A MSP

Part	**Estimate Price**	**Comment**
TTGO LoRa SX1276 ESP32	9 €	incl. antenna
LiPo Battery 1500mAh	3 €	
3D printed case	4 €	
Ublox NEO GPS	12 €	incl. antenna
SD-Card Breakout	2 €	
32 GB SD Card	13 €	
Real Time Clock	1 €	
Temperature & Humidity Sensor	1 €	
Gyro Sensor	1 €	
Total Price	46 €	

The MSP example shown in Table VI is usable for GPS tracking of larger animals plus some added sensors for collecting weather data. In real world deployments with a 750 mAh battery and gathering samples every 15 seconds, the power lasted for about 6 days. With a reasonably lightweight and cheap 1500 mAh battery and a lower sampling rate, we can achieve runtimes over two weeks. The total cost for this setup is around 46 € per device, mainly dominated by the price of the SD card for data logging and the GPS module.

In Table VII, we show a SSP example used for relaying data as well as actively gathering sensor data and processing it on device. Depending on whether a compute stick is used or not, the cost varies between 135 € and 201 €. The cost for a blank relay-only system would be around 65 €. The rest is the cost of the added sensors and the NCS. What this setup is lacking, is a power supply. Depending on the location of deployment, a battery system (20 € - 200 €) and/or a solar panel (50 € - 200 €) could be added if a direct power supply is not available.

TABLE VII
EXAMPLE CONFIGURATION FOR A SSP (W/O POWER SUPPLY)

Part	**Estimate Price**	**Comment**
Raspberry Pi 3	32 €	
TTGO LoRa SX1276 ESP32	9 €	incl. antenna
3D printed case	10 €	
Ublox NEO GPS	12 €	incl. antenna
32 GB SD Card	13 €	
Real Time Clock	1 €	
Temperature & Humidity Sensor	1 €	
Pi Camera	20 €	night vision opt.
Movidius NCS	*66 €*	*optionally*
Thermal Imaging	32 €	
Rain Sensor	1 €	
Soil Moisture	3 €	
PIR Motion Detector	1 €	
Total Price	135 € / *201 €*	

VI. CONCLUSION

In this paper, we have presented a flexible and affordable sensor, computation, and communication platform for environmental monitoring. We provided solutions for static and mobile setups, and integrated mobile devices, such as smartphones, into long-range infrastructureless radio networks. Apart from evaluating typical power requirements of common platforms and sensors, we also included LoRa radio transceivers in our study and evaluated their realistic communication range. Furthermore, to make optimal use of these long-range, low-bandwidth links, we provided a solution to on-device preprocessing of image data using neural compute sticks in an energy efficient and fast way. Finally, we presented the expected costs of the subsystems used in our evaluation. The components developed for our setups, such as the rf95 modem firmware, the BLE modem, Serval LBARD support, and the software customized for our Raspbian distribution, are all released as open source software on github.

In the future, the possibilities of direct device-to-device communication via cheap LoRa addons for smartphones should be used more extensively. This opens possibilities for new apps on mobile devices, such as infrastructureless messaging, or using a phone for sensing and remote control tasks. Furthermore, the usefulness of DTN software directly on the MCUs based on miniDTN should be explored. Finally, the integration of energy harvesting solutions would also increase the flexibility of the system when deployed in remote places.

ACKNOWLEDGMENT

This work is funded by the LOEWE initiative (Hessen, Germany) within the NICER project and the Deutsche Forschungsgemeinschaft (DFG, SFB 1053 - MAKI).

REFERENCES

[1] K. Aberer, S. Sathe, D. Chakraborty, A. Martinoli, G. Barrenetxea, B. Faltings, and L. Thiele, "Opensense: open community driven sensing of the environment," in *ACM SIGSPATIAL Int. Workshop on GeoStreaming*. ACM, 2010, pp. 39–42.

[2] B. Maag, Z. Zhou, and L. Thiele, "W-air: Enabling personal air pollution monitoring on wearables," *Proc. ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, vol. 2, no. 1, p. 24, 2018.

[3] N. Castell, M. Kobernus, H.-Y. Liu, P. Schneider, W. Lahoz, A. J. Berre, and J. Noll, "Mobile technologies and services for environmental monitoring: The Citi-Sense-MOB approach," *Urban Climate*, vol. 14, pp. 370–382, 2015.

[4] M. Gerboles, L. Spinelle, A. Kotsev, M. Signorini, and L. Srl, "Airsenseur: An open-designed multi-sensor platform for air quality monitoring," in *4th Scientific Meeting EuNetAir*, 2015, pp. 3–5.

[5] P. Sikka, P. Corke, L. Overs, P. Valencia, and T. Wark, "Fleck-a platform for real-world outdoor sensor networks," in *3rd. Int. Conf. on Intelligent Sensors, Sensor Networks and Information*. IEEE, 2007, pp. 709–714.

[6] M. T. Lazarescu, "Design and field test of a WSN platform prototype for long-term environmental monitoring," *Sensors*, vol. 15, no. 4, pp. 9481–9518, 2015.

[7] C. Llamas, M. A. González, C. Hernández, and J. Vegas, "Open source hardware based sensor platform suitable for human gait identification," *Pervasive and Mobile Computing*, vol. 38, pp. 154–165, 2017.

[8] J. A. Wirwahn and T. Bartoschek, "Usability engineering for successful open citizen science," in *Free and Open Source Software for Geospatial (FOSS4G) Conference Proceedings*, vol. 15, no. 1, 2015, p. 54.

[9] N. Fröschle, "Engineering of new participation instruments exemplified by electromobility, particulate matter and clean air policy-making," *HMD Praxis der Wirtschaftsinformatik*, vol. 54, no. 4, pp. 502–517, 2017.

[10] C. Pham, A. Rahim, and P. Cousin, "Waziup: A low-cost infrastructure for deploying iot in developing countries," in *Int. Conf. on e-Infrastructure and e-Services for Developing Countries*. Springer, 2016, pp. 135–144.

[11] Y. Mao, J. Wang, B. Sheng, and F. Wu, "Building smartphone ad-hoc networks with long-range radios," in *34th Int. Conf. on Computing and Communications*. IEEE, 2015, pp. 1–8.

[12] L. Baumgärtner, P. Gardner-Stephen, P. Graubner, J. Lakeman, J. Höchst, P. Lampe, N. Schmidt, S. Schulz, A. Sterz, and B. Freisleben, "An experimental evaluation of delay-tolerant networking with Serval," in *IEEE Global Humanitarian Technology Conference (GHTC '16)*. IEEE, 2016, pp. 70–79.

[13] L. Baumgärtner, S. Kohlbrecher, J. Euler, T. Ritter, M. Stute, C. Meurisch, M. Mühlhauser, M. Hollick, O. von Stryk, and B. Freisleben, "Emergency communication in challenged environments via unmanned ground and aerial vehicles," in *Global Humanitarian Technology Conference (GHTC '17)*. IEEE, 2017, pp. 1–9.

[14] P. Lampe, L. Baumgärtner, R. Steinmetz, and B. Freisleben, "Smartface: Efficient face detection on smartphones for wireless on-demand emergency networks," in *24th Int. Conf. on Telecommunications (ICT)*. IEEE, 2017, pp. 1–7.

[15] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," *CoRR*, vol. abs/1512.00567, 2015.

[16] P. Graubner, P. Lampe, J. Höchst, L. Baumgärtner, M. Mezini, and B. Freisleben, "Opportunistic named functions in disruption-tolerant emergency networks," in *ACM International Conference on Computing Frontiers 2018 (ACM CF'18)*, Ischia, Italy, May 2018.